SWP OpenMP: Task 3 Kmeans

Our CUDA Approach

Group 1: Felix Krückel, Luke Hassel, Gerrik Berner

RWTH Aachen University

January 19, 2026

Agenda

K-Means Algorithm Steps

1. **Initialization**: Select k initial centroids (randomly).
2. **Assignment**: Assign each data point to the nearest centroid (Euclidean distance).
3. **Update**: Recalculate centroids as the mean of assigned points.
4. **Repeat**: Iterate steps 2 and 3 until convergence or max iterations.

Pseudocode:

```
Initialize centroids = random(k)
While not converged:
    For point in data:
        closest = argmin(distance(point, c) for c in centr
        assign(point, closest)
    For c in centroids:
        c = mean(assigned_points)
```

Why CUDA?

- Explicit control over memory management (Pinned Memory, Async/Streams).
- Fine-grained kernel optimization (atomicAdd, shared memory potential).
- Efficient Multi-GPU handling with OpenMP + `cudaSetDevice`.

Memory Management

We use **Pinned Memory** on the host for faster transfers and **Streams** for asynchronous execution.

```
// Host Pinned Memory Allocation
double *pinned_x, *pinned_y;
cudaMallocHost(&pinned_x, n * sizeof(double));
cudaMallocHost(&pinned_y, n * sizeof(double));

// Copying to Device (Async)
cudaMemcpyAsync(devices[d].points_x, pinned_x + start,
                devices[d].point_count * sizeof(double),
                cudaMemcpyHostToDevice, devices[d].stream);
```

The Kernel: Nearest Centroid

Each thread handles one point. We use `atomicAdd` to aggregate results locally on the GPU.

```
__global__ void determine_nearest_centroid(...) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < batch_size) {
        // ... find nearest centroid logic [omitted] ...

        // Accumulate results directly in global memory
        atomicAdd(&sum_x[assignment], px);
        atomicAdd(&sum_y[assignment], py);
        atomicAdd(&count[assignment], 1);
    }
}
```

Multi-GPU Strategy

We use OpenMP to spawn one thread per GPU.

```
int num_devices = 4;
omp_set_num_threads(num_devices);

#pragma omp parallel num_threads(num_devices)
{
    int d = omp_get_thread_num();
    cudaSetDevice(d); // Bind thread to GPU

    // Create per-device stream
    cudaStreamCreate(&devices[d].stream);

    // ... Allocate and Compute ...
}
```

Performance Results

Runtime: 66s

Achieved on the large dataset.

Future Work

- **GEMM Approach**:
  - Reformulate distance calculation as Matrix Multiplication: $\|x - c\|^2 = x^2 + c^2 - 2xc$.
  - Leverage Tensor Cores on H100 for maximum throughput.
  - Potential for significant speedup over standard memory-bound kernels.

Thank You!